

Arquitectura de solución

Sistema de Banca por Internet

Documento de diseño

Modelo C4 · Contexto, contenedores y componentes

Ejercicio práctico · Solución Arquitectónica

Alcance: consulta de movimientos, transferencias y pagos

Canales: SPA (web) y aplicación móvil multiplataforma

Nube: Amazon Web Services (sa-east-1 / DR us-east-1)

Identidad: producto OAuth 2.0 existente de la compañía

18 de septiembre de 2026

Elaborado por: Juan Carlos Segovia H.

Diagramas elaborados con diagrams.net (draw.io), plantilla C4.

ÍNDICE

1. Introducción y alcance
2. Entendimiento de los requerimientos
3. Principios de diseño y atributos de calidad
4. Decisiones arquitectónicas (con alternativas evaluadas)
5. Diagrama de Contexto — C4 nivel 1
6. Diagrama de Contenedores — C4 nivel 2
7. Diagrama de Componentes — C4 nivel 3
8. Arquitectura de las aplicaciones front-end y móvil
9. Autenticación OAuth 2.0 e integración con Onboarding facial
10. Capa de integración, API Gateway y microservicios
11. Acceso a datos y mecanismo para clientes frecuentes
12. Diseño de la solución de auditoría
13. Notificaciones e integración con servicios externos
14. Infraestructura en nube, alta disponibilidad, DR y auto-healing
15. Seguridad, monitoreo y excelencia operativa
16. Marco normativo para una entidad financiera
17. Manejo de costos
18. Riesgos, supuestos y evolución
19. Cierre

Figuras: (1) Contexto, (2) Contenedores, (3) Componentes, (4) Auth y Onboarding, (5) Auditoría y cache, (6) Infraestructura AWS, (7) Secuencia de transferencia.

1. Introducción y alcance

BP necesita un canal de banca por internet para que el cliente consulte el histórico de movimientos, transfiera entre cuentas propias y hacia otros bancos, y realice pagos. La compañía ya tiene Core, un sistema de detalle del cliente, un producto de identidad compatible con OAuth 2.0 y un onboarding móvil con reconocimiento facial. Este documento define cómo se articulan esas piezas sin reescribir el legado y sin inventar un IdP nuevo.

El diseño se documenta con el modelo C4 hasta componentes. La infraestructura se modela en AWS porque el presupuesto cubre baja latencia, alta disponibilidad y servicios gestionados de notificación, biometría y observabilidad. No es un catálogo de productos de moda: cada caja existe porque un requerimiento o un atributo de calidad la exige.

Queda fuera de este ejercicio el Core en sí, la contabilidad mayor, el internet banking para tesorería de empresas complejas y el rediseño de sucursal. Sí entra el canal persona natural / PYME ligera, con el mismo backend y dos frentes (web y móvil).

2. Entendimiento de los requerimientos

2.1 Funcionales

ID	Requerimiento	Cómo se cubre
RF-01	Histórico de movimientos	Servicio de Movimientos + read-model. No se golpea el Core en cada scroll.
RF-02	Transferencias propias e interbancarias	Servicio de Transferencias. El tipo decide Core solo o Core + SPI.
RF-03	Pagos	Mismo servicio, adaptador de recaudador. No se abre un silo nuevo.
RF-04	Datos de cliente desde 2 sistemas	Consulta básica → Core. Detalle bajo demanda → sistema complementario (ACL).
RF-05	Notificar movimientos (mín. 2 canales)	SMS (SNS/Twilio) y Push/Email (FCM + SES). Asíncronos.
RF-06	SPA + app móvil multiplataforma	Angular en web. Flutter en iOS/Android. Un BFF por canal.
RF-07	Auth con producto OAuth 2.0 existente	Se configura, no se reimplementa. Flujo Authorization Code + PKCE.
RF-08	Onboarding facial en el móvil	KYC + Rekognition Face Liveness. Alta en el IdP y enrolamiento de huella/Face ID.
RF-09	Login posterior con clave, huella u otro	Password + MFA. Biometría de dispositivo. OTP como step-up.
RF-10	Auditoría de toda acción del cliente	Interceptor + Outbox + almacén append-only + SIEM.
RF-11	Persistencia de clientes frecuentes	Cache-Aside en Redis, invalidación por evento.

2.2 No funcionales (los que de verdad amarran el diseño)

Atributo	Objetivo que asumo para el canal	Mecanismo
Disponibilidad	99.9 % mensual del canal digital	Multi-AZ, health check, auto-scaling, DNS failover
Latencia	Consulta saldo p95 < 800 ms en sa-east-1	Cache, read-model, CloudFront, región LATAM

Atributo	Objetivo que asumo para el canal	Mecanismo
Integridad	Cero doble débito en reintentos de red	Idempotency-Key, saga, outbox
Seguridad	Defensa en profundidad, mínimo privilegio	WAF, mTLS, JWT, KMS, secretos rotados
Auditoría	Trazabilidad de quién hizo qué y cuándo	Log inmutable 10 años / norma local
Recuperación	RPO 15 min / RTO 60 min ante desastre de región	Aurora global / snapshots + stack DR
Evolución	Sumar un recaudador o un canal sin reescribir el Core	Puertos/adaptadores, eventos, bounded contexts

3. Principios de diseño y atributos de calidad

Trabajo con cuatro reglas que me han salvado en sistemas a medida cuando el legado no se toca y el regulador pregunta después:

El Core es sistema de registro, no un API más. Nada de lógica de producto duplicada en el canal. Si el saldo vive en el Core, el canal consulta u ordena, no inventa un mayor paralelo.

Contratos estables hacia adentro, adaptadores hacia afuera. Arquitectura hexagonal (puertos y adaptadores) + Anti-Corruption Layer. El día que el Core cambie SOAP por ISO 20022, se toca un adaptador, no diez servicios.

Lo síncrono es lo que el cliente está mirando; lo demás es evento. La transferencia confirma o reserva en el Core. Aviso, auditoría y proyección de movimientos van por bus. Si SMS se cae, el dinero no se queda colgado.

Seguridad y evidencia no son un módulo al final. Authorizer en el gateway, mTLS este-oeste, outbox de auditoría en la misma transacción de negocio. Si se deja para la fase 2, no se hace.

En términos de ISO/IEC 25010 priorizo: seguridad, fiabilidad, eficiencia de desempeño y mantenibilidad. Usabilidad la resuelven los dos fronts con un BFF que no les filtre el modelo interno. Portabilidad la da el contenedor (ECS/EKS) y Terraform, no un lock-in de funciones propietarias en cada línea de negocio — Rekognition y Cognito sí se usan, pero detrás de un puerto, por si mañana el banco pide otro vendor de KYC.

4. Decisiones arquitectónicas (con alternativas evaluadas)

Cada decisión lleva al menos dos razones y la opción que descarté. No pongo ADR de treinta páginas: en dos horas se documenta lo que un revisor no puede inferir del dibujo.

4.1 Estilo: microservicios acotados, no una malla de treinta APIs

Elegido: pocos servicios alineados a capacidad de negocio (clientes, movimientos, transferencias/pagos) más onboarding, notificaciones, auditoría, productos y límites. Descartado: monolito modular único y ESB clásico.

Por qué microservicios (1): la consulta de movimientos es lectura masiva (fin de mes, nómina) y la transferencia es escritura con consistencia. Escalar las dos juntas es caro y peligroso. (2): un fallo en notificaciones no puede tumbar el débito. Aislamiento de falla.

Por qué no monolito: más simple de operar el día 1, cierto, pero el enunciado ya pide gateway y tres servicios distintos según el tipo de transacción; forzar un solo deploy acopla el ciclo de Core-adapter con el de push notifications. Por qué no ESB: vuelve a centralizar la lógica, añade latencia y es el anti-patrón del que salimos hace una década en integración bancaria.

4.2 Nube: AWS (evaluado contra Azure)

Elegido: AWS, región primaria sa-east-1 (São Paulo) por latencia hacia Ecuador/LATAM, DR en us-east-1. Evaluado: Azure (APIM + Entra ID + Front Door).

Razones para AWS (1): el onboarding facial se cubre con Rekognition Face Liveness sin montar un producto aparte el día uno, y el bus (EventBridge) + colas (SQS) + SMS/Email (SNS/SES) cierran notificaciones y auditoría con el mismo proveedor. (2): API Gateway, WAF, Shield y Direct Connect son un camino conocido para exponer canal y al mismo tiempo llegar al Core on-premise.

Por qué no Azure en este diseño: es perfectamente válido —sobre todo si el IdP corporativo ya fuera Entra ID— pero aquí el producto OAuth es genérico y el diferenciador es KYC + eventos + notificación. Si la fábrica del banco es 100 % Microsoft, se mapea 1:1 (APIM, Service Bus, Azure AD B2C, Face API) sin cambiar el C4. No me caso con el logo, me caso con los puertos.

4.3 SPA: Angular (evaluado contra React)

Elegido: Angular + TypeScript. Evaluado: React.

(1) En un banco el problema no es dibujar un botón, es que veinte desarrolladores no inventen veinte carpetas. Angular impone módulo, DI e inyección de HTTP interceptors (token, correlación, error de negocio). (2) RxJS encaja con saldos que llegan tarde, polling de una SPI asíncrona y cancelación de llamadas cuando el usuario se sale de la pantalla. React lo hace, pero hay que elegir Redux/Query/router y estandarizarlo a mano; bajo norma, prefiero el marco que ya trae la convención.

React se queda como plan B si el banco ya tiene un design system fuerte en esa línea o si quisieran micro-frontends por producto. Para un canal único de persona natural, Angular es más aburrido y por eso mismo más seguro.

4.4 Móvil: Flutter (evaluado contra React Native)

El enunciado pide framework multiplataforma, dos opciones y el porqué. Elegido: Flutter. Evaluado: React Native.

(1) Consistencia visual y de montos. Flutter dibuja con su motor (Skia). En iOS y Android el comprobante, los decimales y la accesibilidad no dependen de que un puente nativo se haya roto en el último OS. En banca eso no es cosmética. (2) Onboarding y biometría. Cámara, liveness y biometría local (Keychain / Android Keystore) tienen plugins maduros y el desempeño en listas largas de movimientos es predecible. Un solo equipo entrega las dos tiendas.

React Native habría tenido sentido si el SPA fuera React y se quisiera compartir lógica de validación/UI kits. Como el web va en Angular, ese argumento se cae. RN sigue siendo una opción seria; no la elijo aquí por rendimiento de UI y por no arrastrar dos runtimes JS (Hermes + el del web) con distinta deuda.

4.5 Identidad: no rehacer el IdP — sí elegir el flujo OAuth

La compañía ya tiene producto OAuth 2.0. Se configura: dos clientes públicos (SPA, móvil), un cliente confidencial (BFF web), scopes (accounts.read, transfers.write, profile.read), redirect URIs, MFA y rotación de refresh.

Flujo recomendado: Authorization Code + PKCE (RFC 7636) para ambos canales. En web, además, patrón BFF. Es el perfil que pide OAuth 2.1 y RFC 8252 para nativas.

Por qué este flujo (1): el code_verifier evita que un interceptado del authorization code en el móvil o en el redirect del SPA se canjee por tokens. (2): permite refresh rotado y revocación; el canal bancario no puede vivir de un access token eterno en memoria del browser.

Descartado — Implicit: el token viaja en el fragmento de URL, queda en historial y logs de proxy, no hay refresh limpio. IETF lo retiró del BCP. Descartado — Resource Owner Password (ROPC): la app ve la clave, no hay consent ni redirección, facilita phishing y rompe MFA del IdP. Solo se toleraba en legados; aquí hay producto corporativo, no hay excusa. Client credentials: sí, pero solo servicio-a-servicio (adaptador → SPI), nunca el usuario. Device code: no aplica (no hay smart TV).

Web: el BFF es cliente confidencial; el navegador se queda con cookie httpOnly + Secure + SameSite y no toca el refresh. Mitiga XSS, que en SPA es el ataque real. Móvil: AppAuth, tokens en Keystore/Keychain, biometría del SO desbloquea el refresh, no un JWT suelto en SharedPreferences.

4.6 Integración: API Gateway + BFF + ACL (no el front hablando con el Core)

(1) El gateway concentra TLS, JWT, cuota y WAF. Si mañana hay un tercer canal (asistente o WhatsApp), no se reabre el Core. (2) El BFF evita el chatty API desde 3G y esconde el modelo. El ACL traduce el legado. Descarté que Angular/Flutter invoquen SOAP del Core: expone el mapa interno, no hay circuit breaker común y el certificado del mainframe no se distribuye a teléfonos.

4.7 Comunicación interna: REST+JSON síncrono y eventos para el resto

(1) REST es el mínimo común que un banco puede operar con el equipo que ya tiene; gRPC ayudaría entre servicios de alta QPS, pero añade toolchain y no es el cuello (el cuello es el Core). (2) EventBridge/SQS desacopla notificación y auditoría. Descarté orquestación síncrona de SMS dentro del POST de transferencia: un timeout de Twilio no puede devolver 500 al cliente con el débito ya hecho. Este-oeste: mTLS. Nada de JWT de usuario reenviado a diestra y siniestra sin audience distinta (token exchange o token interno de servicio).

4.8 Consistencia de transferencias: saga con orquestación + idempotencia

(1) No hay 2PC realista contra Core + SPI. La saga (reserva/débito → red interbancaria → compensar si la red rechaza) es el patrón que encaja. (2) Idempotency-Key de 24 h: el celular reintenta, el banco no cobra dos veces. Descarté event sourcing como modelo de negocio del dinero: el mayor está en el Core. Event sourcing sí lo uso en auditoría, que es otro problema.

4.9 Clientes frecuentes: Cache-Aside, no Write-Through

(1) El Core no se modifica, así que no puede escribir Redis. Write-Through o Write-Behind exigen que el sistema de registro conozca el cache. (2) Cache-Aside deja la política en nuestro servicio: TTL 5 min, LRU, invalidación por evento CustomerUpdated. Read-Through acoplaría el cliente HTTP al cache. Ver sección 11 y figura 5.

4.10 Onboarding facial: Rekognition + liveness (evaluado contra Jumio / FacePhi / Azure Face)

(1) Prueba de vida (liveness) no es opcional: una foto impresa no puede abrir una cuenta. Rekognition Face Liveness cubre eso en la misma nube. (2) El resultado entra como evidencia al expediente y dispara el alta en el IdP; no sustituye al IdP. Jumio/Onfido/FacePhi son mejores si el banco exige vendor especializado de KYC y cobertura de documentos de muchos países; se enchufan en el mismo puerto. Azure Face quedó atrás en este diseño por el resto del stack AWS. En Ecuador, además, el adaptador puede contrastar cédula con fuentes oficiales cuando el banco tenga convenio; eso es política, no un producto más en el C4.

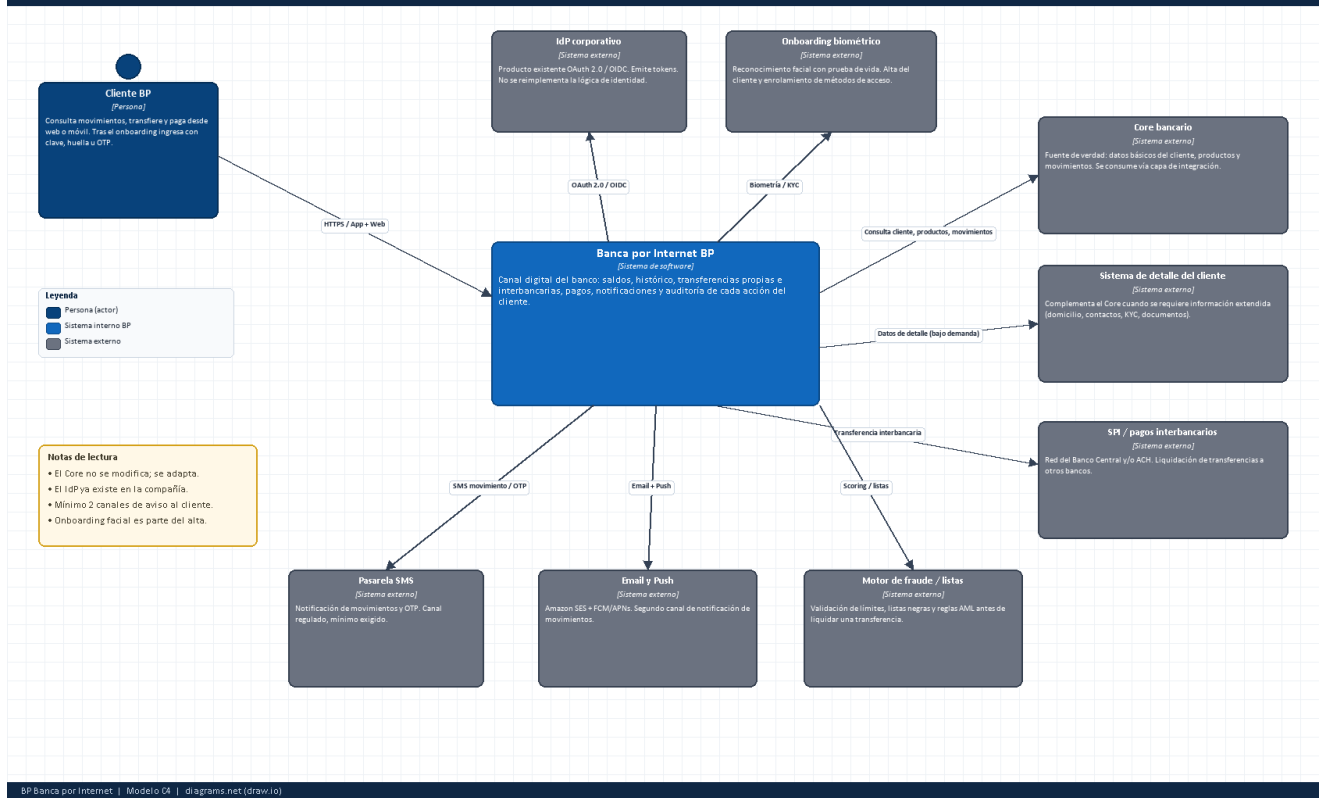
5. Diagrama de Contexto — C4 nivel 1

Audiencia: gente de negocio. El sistema interno es uno: Banca por Internet BP. Alrededor están el cliente y los sistemas que ya existen o que la norma obliga a tocar.

El cliente no “entra al Core”. Entra al canal. El canal habla con el IdP (login), con el onboarding biométrico (alta), con el Core (verdad de saldo y movimiento), con el sistema de detalle (cuando el caso de uso pide más que lo básico), con SPI/ACH (plata que sale del banco) y con dos canales de aviso. El motor de fraude/listas está fuera a propósito: AML no se improvisa dentro del BFF.

Figura 1. Diagrama de Contexto (C4 Nivel 1) — Banca por Internet BP

Audiencia: negocio / no técnico. Muestra el sistema, actores y sistemas externos.

*Figura 1. Diagrama de contexto (C4). Elaborado con diagrams.net / draw.io.*

Lectura rápida: flechas hacia el Core y el CIF son de integración; flechas hacia SMS y Push son notificaciones exigidas por norma; flecha al IdP es autenticación, no un login casero.

6. Diagrama de Contenedores — C4 nivel 2

Audiencia técnica. Aquí aparecen aplicaciones, servicios, bases, cache, mensajería y los componentes cloud sin bajar aún a clases. El ingreso único es API Gateway (WAF delante). Detrás: BFF, IdP existente y servicios. Redis y Aurora son nuestros. Core, CIF, SPI y las pasarelas de aviso siguen siendo externos.

Figura 2. Diagrama de contenedores (C4). SPA, Flutter, gateway, servicios, Redis, Aurora, EventBridge.

Agregué servicios que el enunciado no nombra y que, en operación real, evitan un canal lento o ciego: Onboarding (si se mezcla con transferencias, el KYC se vuelve un lastre), Productos (catálogo cacheado), Límites y fraude (antes del débito, no después), Notificaciones y Auditoría (si viven “en el mismo REST” nadie los escala ni se les pone retención). Los tres originales se mantienen: consulta básica, movimientos, transferencias.

7. Diagrama de Componentes — C4 nivel 3

Bajo el capó del gateway, del BFF y de los tres servicios de negocio. Destaco patrones y protocolos, que es lo que pide el ejercicio: JWT en el borde, mTLS hacia adentro, saga y outbox en transferencias, Cache-Aside en clientes, CQRS de lectura en movimientos.

Detalle técnico: API Gateway, BFF, microservicios, seguridad (mTLS/JWT) y componentes AWS.



Figura 3. Diagrama de componentes (C4). Patrones, protocolos y piezas AWS.

7.1 Segmentación de responsabilidades

Componente	Hace	No hace
API Gateway / WAF	Authn del token, cuota, ruteo, TLS de borde	Reglas de negocio ni SQL
BFF	DTO de canal, cookie de sesión web, correlación	Hablar SOAP con el Core
Servicio Clientes	Básico + cache + enricher de detalle	Transferir dinero
Servicio Movimientos	Histórico y proyección de lectura	Cambiar saldo
Servicio Transferencias	Saga, idempotencia, adaptadores Core/SPI	Enviar SMS en el hilo HTTP
Límites / fraude	Allow / deny / step-up	Persistir el mayor
Notificaciones	Plantilla y reintento SMS/Push/Email	Decidir si la TX es válida
Auditoría	Insertar evidencia inmutable	Corregir una transferencia
ACL Core/CIF	Traducir legado ↔ contrato interno	Exponer el XML del host al Flutter

Cohesión alta dentro de cada bounded context, acoplamiento bajo vía contratos y eventos. Eso es lo que permite “adicionar componentes en el futuro” (un recaudador, un soft-token, un canal de empresas) sin abrir el Core.

8. Arquitectura de las aplicaciones front-end y móvil

8.1 SPA (Angular)

Se sirve por CloudFront + S3, HTTPS only, headers CSP, HSTS y sin secretos OAuth en el bundle. Módulos lazy: login, cuentas, movimientos, transferencias, pagos, perfil. El interceptor HTTP añade correlación y, si se usa BFF, la cookie de sesión; si se usara PKCE puro en el browser (peor), el access token iría solo en memoria, nunca en localStorage. localStorage en banca es regalar la sesión a cualquier XSS.

Estado: el saldo no se “guarda para siempre” en el cliente. Se refresca al volver a la vista y se invalida al confirmar una transferencia. Accesibilidad y montos con formato del locale es-EC (coma decimal, símbolo USD). Logs de UI no llevan PII.

8.2 Móvil (Flutter)

Un codebase, dos binarios firmados. Flavors: dev / staging / prod. El onboarding es un feature module que se apaga por remote config si el producto de KYC está en mantenimiento. Biometría local (huella / Face ID / patrón del SO) no autentica contra el banco por sí sola: desbloquea el material criptográfico que permite usar el refresh. Si el usuario cambia el PIN del teléfono, se pide clave otra vez. Certificate pinning hacia el gateway. Root/jailbreak: se advierte y, en transferencias de alto monto, se bloquea (política del banco, no del framework).

8.3 Lo que comparten y lo que no

Comparten contratos de API (OpenAPI), códigos de error de negocio y el IdP. No comparten UI ni el almacenamiento del token. El BFF puede tener dos perfiles (web/móvil) o dos deploys chicos: en la práctica empiezo con uno y extraigo si el móvil pide agregaciones distintas (widgets de saldo en home). No adelanto esa fractura el día 1.

9. Autenticación OAuth 2.0 e integración con Onboarding facial

Hay dos momentos distintos y mezclarlos es el error típico: el alta (onboarding) y el acceso cotidiano. El reconocimiento facial pertenece al alta y, más tarde, puede usarse como step-up. No es el password de todos los días en un API REST.

Figura 4. Autenticación OAuth 2.0, Onboarding facial y acceso posterior

Flujos recomendados: alta biométrica y Authorization Code + PKCE (SPA y móvil)

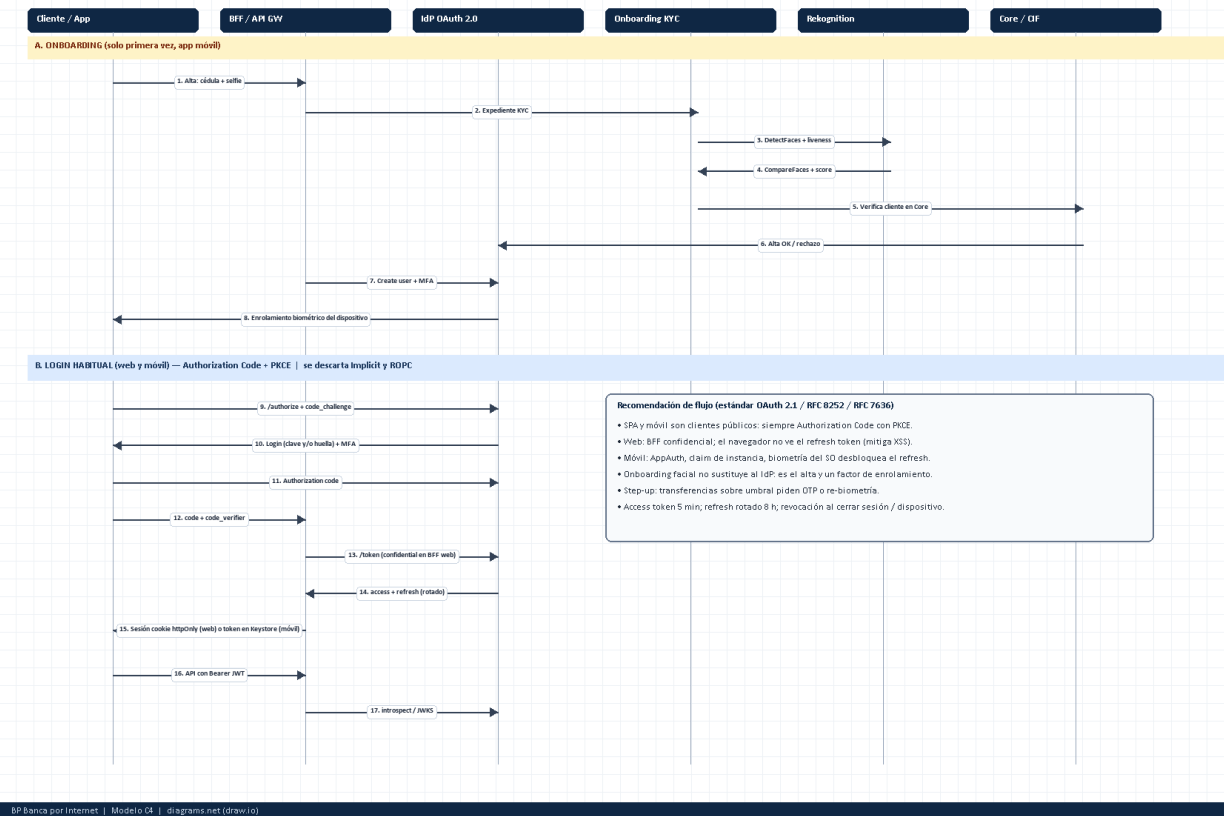


Figura 4. Onboarding biométrico y login Authorization Code + PKCE.

9.1 Onboarding (app móvil)

- El usuario captura cédula (anverso/reverso) y selfie. El SDK de liveness impide foto de foto.
- Onboarding Service manda a Rekognition: DetectFaces, Face Liveness, CompareFaces. Umbral de match lo fija riesgo (ej. 95 %+), no el desarrollador a ojo.
- OCR del documento (Textract o el vendor KYC). Se cruza con Core/CIF: ¿ya es cliente? ¿está en lista de observación?
- Si AML/fraude rechaza, se corta. No se crea usuario “a medias” en el IdP.
- Si ok: el servicio crea el usuario en el IdP corporativo (API de administración del producto, no el flujo de login) y habilita factores: password inicial de un solo uso, MFA (TOTP o SMS) y el enrolamiento de biometría de dispositivo.
- Evidencia (hash de imágenes, score, geolocalización, versión del SDK) se va a auditoría y a un bucket con Object Lock. Las fotos no viajan al Core.

9.2 Acceso posterior

El enunciado pide usuario y clave, huella u otro método. Dejo tres, en este orden de uso real:

Clave + MFA (siempre disponible, inclusive en un teléfono nuevo). Huella / Face ID del dispositivo (después del enrolamiento; el factor es el del SO, atado a este aparato). OTP / push de aprobación como step-up cuando el monto supera el umbral o el beneficiario es nuevo. Un cuarto opcional, si el IdP lo soporta: WebAuthn / FIDO2 en la SPA (llave o passkey). Lo recomiendo porque reduce phishing; no es obligatorio para salir a producción el día 1.

9.3 Configuración que le pediría al dueño del IdP

- PKCE obligatorio en clientes públicos. Redirect URIs exactas, sin wildcards.
- Access token 5 minutos. Refresh rotado, 8 horas o hasta cierre de sesión. Reuse del refresh viejo = revocar la familia (detección de robo).
- MFA condicional: siempre en dispositivos nuevos y en transferencias sobre umbral.

- Scopes finos; el de transferencia no se le da a un cliente de solo consulta.
- Consent y pantalla de login con marca BP, no la genérica del producto, para bajar phishing.
- JWKS cacheado en el authorizer del gateway. Reloj NTP. aud e iss validados, no solo la firma.

10. Capa de integración, API Gateway y microservicios

El enunciado arranca con tres servicios y autoriza a sumar. Los tres se respetan y se rutean en el gateway según el tipo de transacción.

Servicio	Entrada	Salida hacia	Mejora que aporta
Consulta datos básicos	GET cliente	Core (siempre). CIF si el caso pide detalle	Cache-Aside de frecuentes; no hinchar el payload básico
Consulta movimientos	GET histórico	Read-model; Core/CDC por detrás	Paginado, p95 bajo, el Core no pagina 24 meses en línea
Transferencias / pagos	POST comando	Core; SPI si es interbancaria; recaudador si es pago	Saga, idempotencia, límites antes del débito

Servicios extra (rendimiento o respuesta al cliente): Onboarding, Productos, Límites/Fraude, Notificaciones, Auditoría, Adaptadores. Sin límites, el Core se convierte en el antifraude. Sin productos cacheados, cada home pega tres veces al host. Sin notificaciones aparte, el POST de transferencia espera a un SMS.

Figura 7. Secuencia — transferencia propia vs interbancaria

Tercer servicio del enunciado. El tipo de transacción decide el sistema externo a invocar.

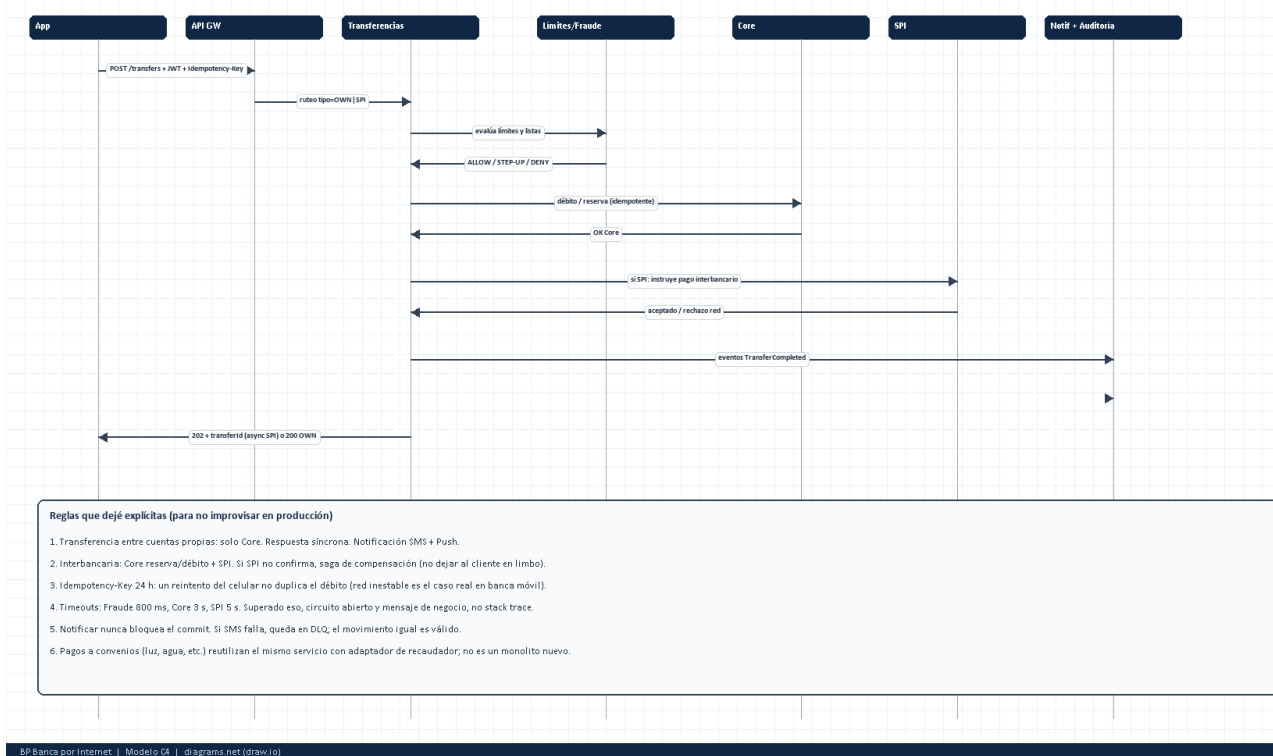


Figura 7. Secuencia de transferencia propia vs interbancaria (el tipo elige el sistema externo).

Protocolo norte-sur: HTTPS 1.2+ / JSON, JWT Bearer. Este-oeste: mTLS. Hacia el Core: lo que el Core hable (SOAP, cola MQ, ISO 20022) envuelto en el ACL, con timeout 3 s y reintento solo si la operación es idempotente. Hacia SPI: correlativo de instrucción, no un POST anónimo.

11. Acceso a datos y mecanismo para clientes frecuentes

No hay una base única “del internet banking” con todo el cliente. Eso duplicaría el Core y a la semana ya estaría desfasada. Cada servicio guarda lo suyo (outbox, expedientes de KYC, políticas de límite). El cliente y el movimiento viven en el legado; nosotros guardamos proyecciones y cache.

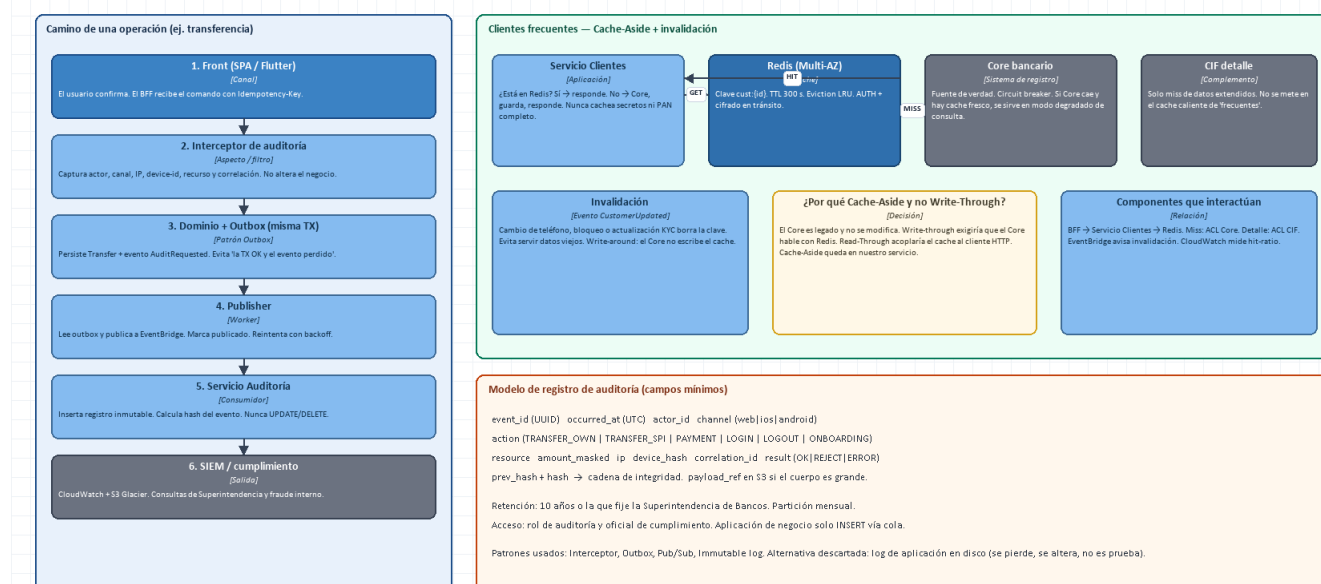
Patrón elegido para frecuentes: Cache-Aside (lazy loading) + invalidación por evento. Componentes que interactúan: BFF → Servicio Clientes → Redis. Miss: ACL Core. Detalle: ACL CIF (fuera del cache caliente). EventBridge publica CustomerUpdated / bloqueo / cambio de teléfono y el servicio borra la clave cust:{id}. CloudWatch mide hit-ratio; si cae del 80 % algo está invalidando de más o el TTL está mal.

Modo degradado: si el Core no responde y hay cache con edad menor a 2 minutos, se sirve la consulta básica con una bandera stale en pantalla. Nunca se autoriza una transferencia contra cache. Transferir pide saldo vivo.

Movimientos: CQRS de lectura. Un read-model (Redis o OpenSearch) se alimenta por eventos o, si el Core no emite, por CDC (AWS DMS) en ventana controlada. El usuario no hace un SELECT de 24 meses contra el host en hora pico.

Figura 5. Auditoría de acciones y persistencia de clientes frecuentes

Patrones: Cache-Aside, Outbox, Event-driven, Interceptores y almacén Inmutable.



BP Banca por Internet | Modelo 04 | diagrams.net (draw.io)

Figura 5. Relación de componentes: Cache-Aside de clientes frecuentes y cadena de auditoría.

Datos en reposo: Aurora cifrado con KMS, Redis AUTH y cifrado en tránsito, S3 de evidencias con Object Lock (WORM) para lo que un auditor no puede ver alterado. No se cachea PAN, CVV ni clave. Mascaramiento en logs.

12. Diseño de la solución de auditoría

La auditoría no es el log de CloudWatch. El log operativo se rota y se pisa. La evidencia de “quién transfirió 1.200 USD el martes a las 21:04 desde este IMEI” tiene que sobrevivir a un deploy y a un abogado.

Patrones que relacionan los componentes:

Interceptor / filtro en el BFF y en cada servicio de comando: captura actor, canal, IP, device hash, acción, recurso, monto enmascarado, correlación y resultado. No conoce de SPI.

Outbox en la misma transacción de negocio: si se confirmó la transferencia y se cayó el proceso antes de publicar, un worker reintentará. Evita el clásico “la plata salió y el auditorio no se enteró” o el inverso.

Pub/Sub (EventBridge + SQS + DLQ) hacia el Servicio de Auditoría. Ese servicio solo inserta. Sin UPDATE, sin DELETE. Hash encadenado opcional (el registro N incluye el hash de N-1) para detectar recortes. Partición mensual. Retención 10 años o la que fije Superintendencia.

Inmutable log / WORM: RDS dedicado o, si el volumen y la inmutabilidad lo justifican, QLDB o S3 Object Lock. Réplica a la región DR. Lectura para oficial de cumplimiento y para export a SIEM (GuardDuty / herramienta del SOC). La aplicación de negocio no tiene SELECT libre sobre la tabla de auditoría.

Descarté “un trigger en la base del Core” porque no cubre login, onboarding ni intentos fallidos, y porque el Core no es nuestro. Descarté logstash en el disco del contenedor: el contenedor se muere y se llevó la prueba.

13. Notificaciones e integración con servicios externos

La norma pide avisar movimientos. Dejo dos canales mínimos y un tercero de cortesía:

Canal	Tecnología	Uso
SMS	Amazon SNS o Twilio (doble proveedor si el banco ya tiene contrato)	Movimiento, OTP, alerta de login nuevo. El que llega sin datos.
Push	FCM (Android) + APNs (iOS) vía SNS	Aviso en la app, bajo costo, segundo canal.
Email	Amazon SES	Comprobante y resumen. No es el canal de alerta inmediata.

Plantillas versionadas, no concatenar strings con el monto (un bug de formato no puede mandar el SMS equivocado). Registro de entrega y de baja (opt-out de marketing; el aviso de movimiento no es marketing y no se apaga). PII mínima en el cuerpo: “movimiento en cuenta **1234 por USD 50” no “transferencia a Juan Pérez cédula...”.

Integraciones externas y cómo no acoplarse

- Core: adaptador + circuit breaker + timeout. Nunca desde el front.
- Sistema de detalle: misma idea, llamado solo cuando el caso de uso lo pide (extracto, actualización de domicilio).
- SPI / ACH / BCE: adaptador propio, conciliación T+0/T+1, no reutilizar el cliente HTTP del Core.
- IdP: configuración, JWKS, API de alta de usuario para onboarding.
- KYC biométrico: puerto. Hoy Rekognition, mañana FacePhi, el dominio no se entera.
- Fraude/listas/AML: decisión allow-deny antes del débito. Timebox 800 ms; si no responde, política fail-closed en alta cuantía y fail-open controlado en consulta.

14. Infraestructura en nube, alta disponibilidad, DR y auto-healing

Nada de una sola AZ “porque es más barato”. Un canal de banco que se cae un viernes de nómina sale más caro que el Multi-AZ. El diagrama 6 es el mapa; aquí van los números que yo firmaré en un comité.

Figura 6. Infraestructura AWS — HA, DR, seguridad, monitoreo y auto-healing

Región primaria sa-east-1 (São Paulo) por latencia LATAM. DR us-east-1. Multi-AZ.

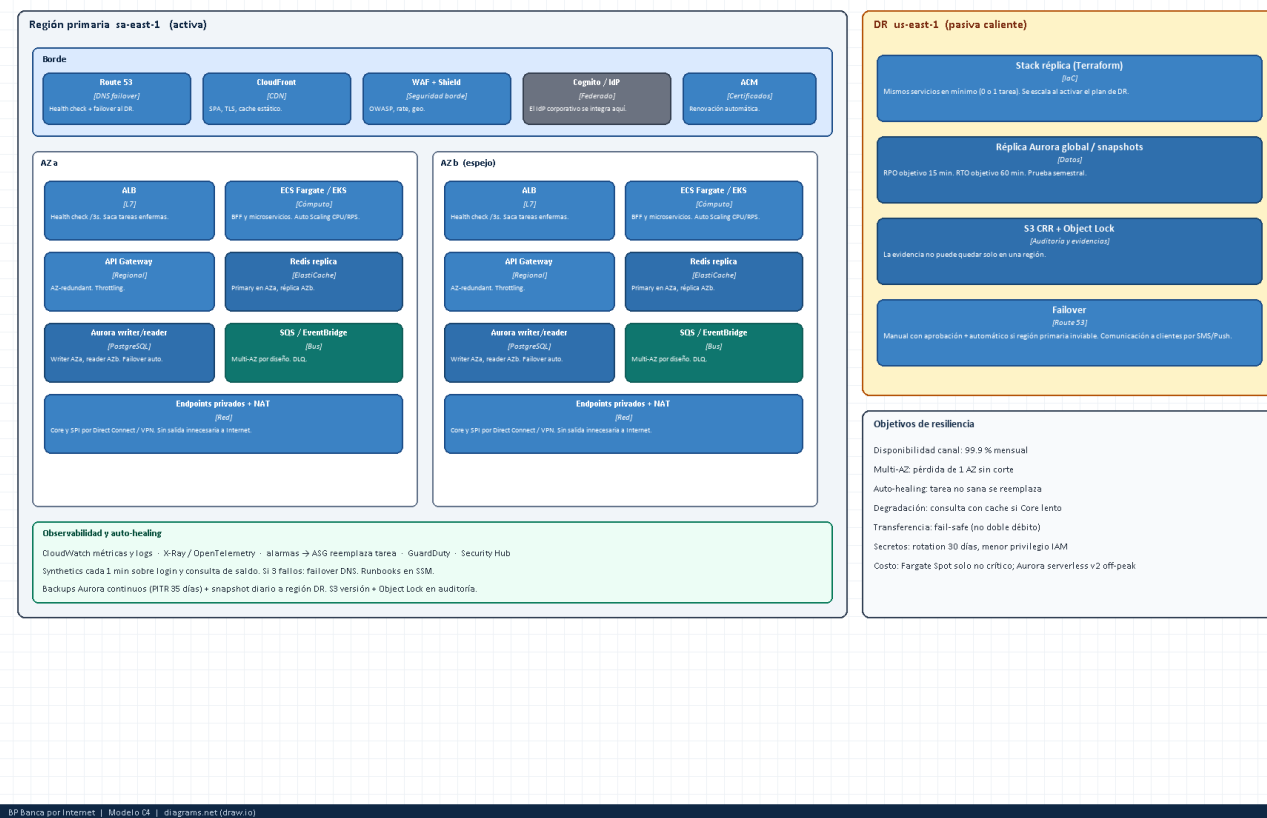


Figura 6. Infraestructura AWS: borde, dos AZ, observabilidad y DR pasivo caliente.

14.1 Alta disponibilidad

API Gateway, ALB, ECS Fargate (o EKS si el banco ya tiene malla) y Aurora Multi-AZ. ElastiCache Redis con réplica. SQS/EventBridge ya son multi-AZ. Health check cada 3 s; si la tarea no responde, el orquestador la reemplaza (auto-healing). Auto Scaling por CPU y por RPS en transferencias. El SPA está en CloudFront: la caída de un origen no apaga la UI, aunque las APIs sí se noten.

14.2 Tolerancia a fallos

Circuit breaker hacia Core, CIF y SPI. Bulkhead: el pool de conexiones del adaptador Core no es el mismo que el de notificaciones. Timeouts explícitos (figura 7). Cola con DLQ y alarma; un mensaje en DLQ no se silencia. Idempotencia de lado a lado. Si Redis cae, el servicio de clientes va directo a Core (más lento, vivo). Si el bus cae, la transferencia igual confirma y el outbox reintenta: la evidencia se atrasa, el dinero no se inventa.

14.3 Recuperación ante desastres

Región DR us-east-1, stack Terraform idéntico en mínimo (warm). Aurora Global o snapshots replicados. S3 CRR con Object Lock para auditoría. RPO 15 min, RTO 60 min. Failover DNS en Route 53, con runbook: no es un botón que un desarrollador aprieta solo a las 3 a.m. Prueba semestral con acta. Comunicación a clientes por SMS/Push (“canal en contingencia”). El Core on-prem tiene su propio DR; si el Core muere, el canal no “inventa saldos”: se pone en modo solo consulta degradada o se cierra transferencia, según el BCP del banco.

14.4 Baja latencia

sa-east-1, CloudFront, cache de frecuentes, read-model de movimientos, keep-alive y payloads chicos en el BFF. Direct Connect / VPN de baja latencia al datacenter del Core. No pongo Lambda en el camino caliente de la transferencia si un cold start va a sumar 400 ms; Fargate caliente es más predecible. Lambda sí en workers de notificación y en el drenado del outbox.

15. Seguridad, monitoreo y excelencia operativa

15.1 Seguridad (defensa en capas)

- Borde: WAF (OWASP, rate, reputación) + Shield. TLS 1.2+ (1.3 preferido). HSTS.
- Identidad: OAuth 2.0 / OIDC como ya se describió. MFA. Step-up.
- Este-oeste: mTLS, security groups por servicio, sin RDS público, endpoints privados.
- Secretos: AWS Secrets Manager, rotación 30 días, IAM de menor privilegio. Nada en el repo.
- Datos: KMS, cifrado en reposo y tránsito, enmascaramiento, tokenización si algún día entra tarjeta (PCI).
- App: OWASP ASVS nivel 2 como piso. Validación de entrada. Idempotencia. Headers CSP en la SPA.
- Móvil: pinning, almacenamiento en Keystore, detección de root como señal de riesgo.
- Detección: GuardDuty, Security Hub, CloudTrail org-wide. Alertas al SOC, no al grupo de WhatsApp del equipo.

15.2 Monitoreo

Tres señales, no un dashboard bonito: latencia p95 de consulta y de transferencia, tasa de error de negocio vs 5xx, hit-ratio de Redis y profundidad de DLQ. Trazas con OpenTelemetry / X-Ray y un correlation id que nace en el front y llega al adaptador del Core. Logs estructurados JSON, sin PII. Synthetics cada minuto: login + saldo. Si fallan tres veces, pagina quien está de guardia y se evalúa failover.

Métricas de negocio (no solo infra): transferencias aceptadas vs rechazadas por fraude, onboardings abandonados en liveness, tiempo a primer movimiento. Eso es excelencia operativa: el canal se opera como producto, no como un IIS que “hace ping”.

15.3 Auto-healing y operación

ECS reemplaza tareas no sanas. Aurora failover automático en la AZ. Runbooks en SSM. Despliegue blue/green o canary en el gateway (el ruteo por transacción permite canary de un servicio sin tocar los otros). Feature flags para apagar onboarding o SPI si el proveedor está en mantenimiento, sin redeploy. IaC en Terraform, cuentas AWS separadas (dev / test / prod), pipeline con SAST, SCA y DAST en staging. No se despliega a prod con un click desde el laptop.

16. Marco normativo para una entidad financiera

BP no está ubicada en el enunciado, pero el ejercicio pide normativa de entidades financieras. Trabajo con el marco que aplico en Ecuador y con estándares internacionales que un banco extranjero o un regulador local van a reconocer. Si BP opera en otra jurisdicción, se sustituye la ley local; PCI, ISO y OAuth no se sustituyen.

Norma / marco	Qué obliga en este diseño
Ley Orgánica de Protección de Datos Personales (LOPD, Ecuador) y su reglamento	Base de legitimación, minimización, consentimiento donde aplique, derechos ARCO, encargados (AWS, SMS), notificación de incidentes, no usar el detalle del cliente “porque está ahí”.
Superintendencia de Bancos del Ecuador — control interno, seguridad de la información y continuidad	BCP/DR con RTO-RPO, pistas de auditoría, segregación de funciones, pruebas, informe a directorio. El canal digital es proceso crítico.
UAFFE / prevención de lavado de activos y financiamiento del terrorismo	KYC en onboarding, listas, umbrales, congelamiento, no “agilizar” el alta saltándose AML. El motor de fraude/listas no es opcional.
Banco Central del Ecuador — sistema de pagos (SPI) y liquidación	Transferencia interbancaria por riel regulado, conciliación, horarios de cámara. El adaptador SPI existe por esto, no por gusto.

Norma / marco	Qué obliga en este diseño
Ley de Comercio Electrónico, Firmas y Mensajes de Datos (Ecuador)	Mensaje de datos, consentimiento, conservación. El comprobante de transferencia es evidencia.
PCI-DSS (si hay datos de tarjeta en pagos)	No almacenar CVV. Segmentación. Si solo se paga a convenios vía token del recaudador, se reduce el alcance. No meter PAN en Redis “por si acaso”.
ISO/IEC 27001 y 27002; ISO 22301 (continuidad)	SGSI, gestión de riesgos, controles. Encaja con IAM, cifrado, registro, DR.
NIST SP 800-63 (identidad digital) y OWASP ASVS / MASVS	Niveles de autenticación, AAL2 para movimiento de fondos, biometría como factor, guía de app móvil.
OAuth 2.0 / OIDC + RFC 8252, RFC 7636, OAuth 2.1 BCP	El flujo que elegí no es preferencia estética; es el perfil que sobrevive una revisión de seguridad.
SWIFT CSP (si aplica mensajería internacional)	No exponer credenciales SWIFT al canal. El adaptador de pagos internacionales, el día que exista, vive en zona restringida.

Privacidad por diseño: el sistema de detalle no se consulta en el home. El SMS no lleva nombre del beneficiario. La auditoría guarda device hash, no la foto de la cara. Las imágenes de onboarding tienen retención más corta que el registro del alta, salvo que el jurídico diga lo contrario por reclamo de identidad.

17. Manejo de costos

Hay presupuesto para baja latencia; no por eso dejo todo en instancia 24x7 de más. El costo en un canal bancario se va en tres sitios: el Core (ya existe), el tráfico de notificación y el cómputo de lectura. Ahí aprieto.

Palanca	Qué hago	Qué no hago
Cómputo	Fargate con autoscaling. Spot solo en workers no críticos (notificación, reportes). Lambda en outbox.	Un EKS de 12 nodos m5.4xlarge “porque Kubernetes” el día 1.
Datos	Aurora Serverless v2 o instancias right-sized. TTL agresivo en Redis. S3 IA / Glacier en evidencias viejas.	Cachear el universo de clientes “por si se vuelven frecuentes”.
Red / borde	CloudFront (el estático casi no cuesta). Direct Connect se justifica por latencia y por no salir a Internet hacia el Core.	NAT gigante con todo el tráfico de pod a pod.
KYC	Rekognition por onboarding exitoso, no por cada reintento de cámara en el cliente (se corta en el SDK).	Reprocesar el video 4 veces “para estar seguros”.
Compromiso	Savings Plans / RI a 1 año cuando el baseline de prod sea estable (3 meses de métrica).	Comprar 3 años el primer viernes.
Observabilidad	Logs con retención corta en hot, larga en S3. Sampling de trazas en consulta, 100 % en transferencia.	X-Ray al 100 % de GET de catálogo.
Licencias	IdP ya pagado. Flutter y Angular sin royalty. Aurora vs Oracle nuevo: no meto Oracle en el canal.	Comprar un ESB o un BPM para orquestar tres POST.

FinOps mínimo: etiqueta por servicio y ambiente, presupuesto con alarma al 80 %, revisión mensual de “huérfanos” (NAT, snapshots, IPs). El costo de un incidente de doble débito o de una multa LOPDP no cabe en el Excel de AWS; por eso no ahorro en auditoría ni en MFA.

18. Riesgos, supuestos y evolución

18.1 Supuestos

- El producto OAuth 2.0 de la compañía soporta PKCE, MFA, clientes confidenciales y alta de usuario por API. Si no soporta PKCE, se actualiza o se pone un Brokering (p. ej. delante con un adaptador); no se vuelve a ROPC.
- El Core expone, aunque sea de forma tosca, consulta de cliente/productos/movimientos y un comando de débito/transferencia interna. Si solo hay pantalla 3270, este diseño no arranca: primero un wrapper.
- Existe riel interbancario (SPI/ACH) con contrato y ambiente de certificación.
- Hay conectividad privada datacenter ↔ AWS.
- El onboarding facial está autorizado por jurídico y por protección de datos (biometría es dato especial).

18.2 Riesgos que dejaría en el registro

Riesgo	Mitigación
Latencia o ventana caída del Core	Cache degradada en consulta, circuit breaker, no transferir en ciego, mensaje de negocio.
Doble instrucción a SPI por reintento	Idempotency-Key + correlativo de red + conciliación.
Fraude con selfie (spoofing)	Liveness, umbral, device intelligence, step-up, listas.
XSS en la SPA robando sesión	BFF + cookie httpOnly, CSP, sin token en localStorage.
Vendor lock-in de Rekognition o SNS	Puertos/adaptadores; segundo proveedor SMS.
Costo de SMS en campañas de “notificar todo”	Plantillas cortas, umbral, push primero si la app está instalada.
El sistema de detalle se vuelve chatty	Nunca en el home; batch o pantalla de perfil; presupuesto de llamadas.

18.3 Cómo crece sin reescribir

Un nuevo recaudador es un adaptador. Un canal WhatsApp es otro BFF atrás del mismo gateway. Una cuenta de inversión es otro bounded context que publica eventos; el home del Flutter suma un widget. Soft-token o FIDO2 se configuran en el IdP. Eso es exactamente “elementos reusables y cohesionados”. No adelanto event sourcing del mayor ni service mesh el primer mes.

19. Cierre

La solución cubre el enunciado sin teatro: dos fronts (Angular y Flutter, con la alternativa de React y React Native explicada), identidad con el producto OAuth existente y el flujo Authorization Code + PKCE, onboarding facial como alta y no como parche, tres servicios de negocio más los que hacen falta para no mentir en latencia ni en evidencia, cache-aside de clientes frecuentes, auditoría inmutable, dos canales de notificación, AWS multi-AZ con DR, y el marco normativo con el que un banco en la región no puede hacerse el distraído.

Si algo hay que defender en una mesa no es el color de las cajas: es que el Core no se expone, que el dinero no se duplica en un reintento, que el SMS no decide la transacción y que cada decisión de este papel tiene alternativa evaluada. El resto es disciplina de implementación.